

## 04 - Transactions and SMS Sync

### Feature objective

Build the Android transaction engine: fast manual entry, local SMS scanning/parsing, normalized sync, deduplication, transfer/card-payment handling, review queue, categories, and corrections.

This is one of the highest-priority MVP features.

### Dependencies

- Foundation/App Shell
- Auth
- Accounts
- local SQLite
- local Expo Kotlin SMS module
- backend transaction endpoints

### In scope

- transaction list
- transaction detail/edit
- manual transaction creation
- search/filter
- SMS permission state
- foreground/on-open SMS scan
- first 60-day scan
- incremental scan using cursor/timestamp
- deterministic local parser
- low-confidence review queue
- normalization
- local dedupe
- backend dedupe
- self-transfer recognition
- credit-card payment handling
- category correction rules
- outbox for offline manual/SMS transactions
- sync status

### Out of scope

- background continuous SMS receiver
- full bank statement PDF parser
- Account Aggregator
- OCR
- live bank sync

### Routes

```
(tabs)/transactions.tsx
transaction/new.tsx
transaction/review.tsx
transaction/[id].tsx
settings/sms.tsx
```

### Feature folders

```
features/transactions/
|-- screens/
```

```

|-- components/
|-- hooks/
|-- services/
|-- forms/
|-- utils/
|-- schemas/
`-- types.ts

features/sms-sync/
|-- hooks/
|-- parser/
|   |-- templates/
|   |-- normalize.ts
|   |-- classify.ts
|   `-- confidence.ts
|-- services/
|-- database/
|-- sync/
|-- utils/
`-- types.ts

```

Native module:

```
modules/finance-sms/
```

## Native module interface

Minimal Kotlin/Expo module contract:

```

checkPermission(): Promise<'granted' | 'denied' | 'blocked'>
requestSmsPermission(): Promise<'granted' | 'denied' | 'blocked'>
getSmsSince(timestampMs: number): Promise<RawSmsRecord[]>

```

Do not put financial parsing logic in Kotlin unless Android platform behavior requires it. Keep parsing/testability in TypeScript.

## SMS privacy rule

- scan on device
- classify financial messages locally
- ignore non-financial messages
- do not upload raw personal inbox history
- sync normalized financial transaction records
- store only the minimum raw snippet locally when essential for review/debugging; prefer hashed/template meta-data rather than full message content
- never log raw SMS in production logs

## Sync lifecycle

On app foreground/resume:

```

read lastSmsScanAt
  v
permission granted?
  |-- no -> no scan
  `-- yes
    v
get SMS since cursor
  v
local financial classification

```

```

      v
parse + normalize
      v
local dedupe
      v
store candidates/outbox
      v
batch sync normalized records
      v
backend dedupe/reconcile
      v
update lastSuccessfulSyncAt

```

Throttle foreground checks so switching screens does not trigger repeated scans.

## Normalized candidate

Example concept:

```

{
  "amount": 650,
  "currency": "INR",
  "direction": "DEBIT",
  "merchant": "SWIGGY",
  "paymentMethod": "UPI",
  "accountLast4": "3812",
  "occurredAt": "2026-08-30T14:41:00+05:30",
  "balanceAfter": 18331.23,
  "source": "SMS",
  "confidence": 0.98,
  "externalReference": "optional"
}

```

## Parser strategy

Priority:

1. known sender/template rules
2. deterministic regex/token parser
3. merchant/account normalization
4. confidence score
5. unknown template fallback for backend/AI only when justified

Do not send every SMS to AI.

Parser should extract when available:

- amount
- debit/credit
- merchant/payee
- account/card last4
- payment method
- transaction/reference ID
- date/time
- balance-after
- sender/template family

## Confidence policy

High confidence:

- auto-import/sync

Medium/low confidence:

- place in Needs Review

Review actions:

- Confirm
- Edit
- Ignore

When user corrects a merchant/category mapping, ask whether to apply it to future matching transactions when appropriate.

## Deduplication

Strong identifiers:

- UTR/RRN/reference ID
- card/bank transaction ID

Fallback fingerprint:

`account + direction + amount + timestamp bucket + normalized merchant`

Backend owns canonical reconciliation.

One real transaction may have multiple provenance sources later:

`SMS + PDF + Account Aggregator`

User should see one transaction, not three duplicates.

## Self-transfers

Detect likely transfer between owned accounts.

High confidence:

- mark as transfer automatically
- exclude from income/expense totals

Ambiguous:

- review

Never delete the transaction just because it is excluded from spending.

## Credit-card accounting

Credit-card purchase:

`expense + increases credit-card liability`

Credit-card bill payment:

`bank transfer/liability settlement`

`not a second expense`

UI should clearly label card payment as transfer/payment.

## Categories

Initial compact taxonomy:

- Food
- Transport
- Shopping

- Bills
- Rent
- EMI
- Healthcare
- Entertainment
- Education
- Salary
- Transfer
- Investment
- Travel
- Subscriptions
- Other

User correction overrides AI/rules.

## Transaction list

Use FlashList.

Header:

- last sync status
- search
- filter chips
- monthly spending/income summary

Filter examples:

- All
- Needs review
- Expense
- Income
- Account
- Category
- Date

Do not render excessive badges on every row.

## Manual transaction

Amount-first flow:

Amount

Expense | Income | Transfer

Merchant/description

Category

Account

Date

Note optional

The common path should take under 10 seconds.

If offline:

- save to SQLite outbox
- show pending sync state
- sync later with idempotency key

## Transaction detail

Show:

- amount

- merchant
- category
- type
- account
- date/time
- payment method
- source/provenance
- masked reference
- balance-after if present
- estimated/confirmed status
- impact on monthly spending

Actions:

- edit category
- edit merchant
- add note
- exclude from spending
- mark/repair transfer where allowed

## AI categorization on backend

After deterministic merchant/category rules:

- batch only unknown unique merchants
- categorize repeated merchant once
- persist user-correction rule
- use BullMQ so transaction save/sync is not blocked

## API requirements

Conceptual:

```
GET    /transactions
POST   /transactions
GET    /transactions/:id
PATCH /transactions/:id
POST   /transactions/sync
POST   /transactions/:id/review
GET    /transactions/review
GET    /transaction-categories
```

Sync endpoint should support batch idempotency and return per-item reconciliation result.

## Local database

Suggested tables:

```
sms_candidates
transaction_outbox
sync_state
merchant_rules_cache
```

Important metadata:

```
lastSmsScanAt
lastSuccessfulSyncAt
parserVersion
```

## Step-by-step implementation

1. Build manual transaction form + backend create.

2. Build transaction list/detail.
3. Build category/filter utilities.
4. Scaffold native SMS module.
5. Add permission UX/settings.
6. Implement deterministic parser with fixtures.
7. Add first 60-day scan.
8. Add incremental foreground scan.
9. Add local candidate/outbox persistence.
10. Add batch backend sync.
11. Add dedupe/reconciliation handling.
12. Add self-transfer logic.
13. Add credit-card payment behavior.
14. Add review queue.
15. Add user merchant/category rules.
16. Add offline pending-sync UX.

## Acceptance criteria

- manual transaction works online and offline
- 60-day first scan works when SMS permission is granted
- non-financial SMS does not become uploaded transaction data
- routine foreground scan only reads new messages
- high-confidence transaction imports without blocking UI
- low-confidence item appears in Review
- duplicate sync does not create a duplicate canonical transaction
- self-transfer is excluded from spending
- credit-card purchase + bill payment does not double-count spending
- user correction overrides future automatic categorization for matching merchant when rule is enabled
- sync failure preserves local pending data

## Tests

Highest-priority mobile tests in the project:

- parser fixtures for many bank/payment templates
- malformed SMS
- debit vs credit
- balance-after extraction
- dedupe fingerprints
- strong reference dedupe
- self-transfer matching
- credit-card payment behavior
- review transitions
- outbox retry/idempotency
- foreground scan cursor
- permission denied/blocked